什么是索引?

索引是一种帮助存储引擎快速查找数据的数据结构

索引的分类?

按[数据结构]分类：B+Tree索引、Hash索引、Full-text索引
按[物理存储]分类：聚簇索引(主键索引),二级索引(辅助索引)
按[字段特性]分类：主键索引、唯一索引、普通索引、前缀索引
按[字段个数]分类：单列索引、联合索引

`explain` + `sql` --> 返回一份**执行计划报告**

key字段为null则表示没有使用索引

type字段描述了找到所需数据时使用的扫描方式是什么

执行效率从低到高

扫描方式	**type值**
全表扫描	All
全索引扫描	index
索引范围扫描	range
非唯一索引扫描	ref
唯一索引扫描	eq_ref
结果只有一条的主键或者唯一索引扫描	const

const：

当查询条件是**主键（PRIMARY KEY）*或*唯一索引（UNIQUE KEY）**，并且查询条件是精确匹配（如 `WHERE id = 123`）时，数据库能确定最多只会返回一条记录（或没有记录）。这种情况下，查询效率极高（常量扫描），`type` 字段就会显示为 `const`。

B+Tree 是一种多叉树，叶子节点才存放数据，非叶子节点只存放索引，而且每个节点里的数据是按主键顺序存放的。每一层父节点的索引值都会出现在下层子节点的索引值中，因此在叶子节点中，包括了所有的索引值信息，并且每一个叶子节点都有两个指针，分别指向下一个叶子节点和上一个叶子节点，形成一个双向链表。

B + 树相比 B 树和二叉树查询效率更高，核心是其非叶子节点仅存索引、叶子节点链式相连且数据仅存于叶子层，既大幅降低磁盘 IO 次数，又让范围查询无需回溯，同时规避了二叉树易退化为链表的单路查询低效问题。

非叶子节点仅存索引 + 数据仅存叶子层，能最大化单个节点的存储密度，直接让树的层数剧减，大幅减少磁盘 IO 次数；叶子节点链式相连，让范围查询无需回溯父节点，一次遍历即可完成，彻底解决 B 树范围查询的低效问题。

存储引擎逐层向下定位

回表

通过非主键索引的B+树找到叶子节点中的主键值，再用该主键值去主键索引的B+树逐层定位，找到对应叶子节点的完整行数据，这两次索引查询过程就叫做回表

覆盖索引

非主键索引的 B + 树叶子节点直接存储查询所需的全部字段值，无需回表查询主键索引即可得到结果，这就是覆盖索引。

联合索引

1. 需要遵循最左匹配原则，也就是按照最左优先的方式进行索引的匹配
2. 如果有联合索引（a,b,c）
   那么b和c全局无序，局部有序
3. 并不是查询过程用到了联合索引，就代表联合索引中的所有字段都用到了联合索引进行索引查询
4. 联合索引的最左匹配原则，在遇到范围查询（如 >、<）的时候，就会停止匹配，也就是范围查询的字段可以用到联合索引，但是在范围查询字段的后面的字段无法用到联合索引。注意，对于 >=、<=、BETWEEN、like 前缀匹配的范围查询，并不会停止匹配

索引下推

索引下推是 MySQL 将联合索引中可筛选的**非主键条件**下推至存储引擎层，在索引遍历阶段提前过滤数据，减少回表的行数和 IO 开销。

Extra：Using index conditon表示用到了索引下推

索引区分度

索引区分度是指索引列中唯一值的占比，占比越高区分度越好，能让数据库通过索引快速缩小查询范围、减少扫描行数。

什么时候适合用索引?

1. 有唯一性限制的字段
2. 经常用于where、order by和group by条件查询的字段

什么时候不需要创建索引?

1. where、order by和group by条件里用不到的字段，因为索引的价值是快速定位，如果起不到定位的价值就不需要创建索引，索引也是会占用物理空间的
2. 表中数据过少
3. 表中存在大量重复数据
4. 经常更新的字段不用创建索引，索引字段被频繁修改，由于要维护B+Tree的有序性，那么就需要频繁的重建索引，这会影响数据库性能

有什么优化索引的方法?

前缀索引优化

前缀索引就是使用字段的前几个字符建立索引，可以减小索引项的大小，从而增加一个索引页中存放的索引值数量，提高索引的查询效率

覆盖索引优化

覆盖索引就是在非主键索引B+树的叶子节点中直接存储查询所需的所有字段值，无需回表查询主键索引即可得到结果

1. 核心核心：设计“宽”联合索引
实现覆盖索引最直接的手段是根据查询需求，将原本不在索引中的字段，加入到联合索引中。

做法： 如果业务经常执行 `SELECT a, b FROM table WHERE c = 10`，那么不要只给 `c` 加索引，而是建立 `(c, a, b)` 的联合索引。

原理： 此时非主键索引的叶子节点已经包含了 `a` 和 `b` 的值，MySQL 发现索引树里的信息够用了，就不会再去主键索引（聚簇索引）里找整行数据了。

2. 严控查询字段（拒绝 `SELECT *`）
开发规范中强调“禁止 `SELECT *` ”的本质原因之一就是为了配合覆盖索引。

做法： 只查询必要的字段。即使你建立了 `(name, age)` 的联合索引，但如果你写了 `SELECT *`，由于索引里没有 `address` 等其他字段，数据库被迫必须回表。

话术： “我会要求开发团队在编写 `SQL` 时精准匹配索引字段，确保查询列是索引列的子集。”

3. 利用执行计划确认（EXPLAIN）
空谈优化没用，必须通过工具验证。

做法： 使用 `EXPLAIN` 查看 `SQL` 的执行计划。

关键指标： 观察 `Extra` 列。如果显示 `Using index`，说明覆盖索引已生效；如果显示 `Using index condition`，说明只用到了索引过滤但仍需回表。

主键索引最好是自增的

- 如果我们使用自增主键，那么每次插入一条新纪录，都是追加操作，不需要移动数据，这种插入数据的方法效率很高
- 如果我们不使用自增主键，就有可能会插入到数据页中间的某个位置，如果当前页满，就会不得不移动其他数据，甚至需要从一个页面复制数据到另一个页面，这种情况就叫做页分裂。页分裂还有可能会造成大量的内存碎片，导致索引结构不紧凑，从而影响查询效率

索引最好设置为NOT NULL

1. 索引列存在NULL会导致优化器在索引优化的时候更加复杂，更加难以优化
2. NULL值是一个没有意义的值，但他会占用物理空间，所以会带来存储空间的问题

防止索引失效

- 使用左或者左右模糊匹配时
- 在查询条件中对索引列进行了计算、函数或类型转换操作
- 联合索引没有遵循最左匹配原则
- where子句中or前的条件列是索引列，or后的条件列不是索引列

执行SELECT语句的流程

连接器

先连接Mysql服务

1. 与客户端进行TCP三次握手建立连接
2. 校验用户名和密码
3. 如果校验通过，则读取该用户的权限，后面有关的权限逻辑判断都会基于此时读取到的权限

查询缓存

连接器的工作完成后，客户端就可以向Mysql服务发送SQL语句了，Mysql收到语句后，会解析出语句的第一个字段，看看是不是SELECT，是的话就会查询缓存

Mysql8.0将查询缓存删掉了

关闭查询缓存：query_cahce_type设置为DEMAND

解析SQL

在正式执行SQL查询语句之前，Mysql会先对SQL语句进行解析，这个工作由解析器完成

解析器会做两件事情，一个是词法分析，另一个是语法分析

词法分析：根据输入的字符串解析出关键字出来

语法分析：语法分析器根据词法分析的结果和语法规则，判断sql语句是否符合sql语法规范，符合则构建sql语法树

执行SQL

每条SELECT查询语句流程都可以分为以下三个阶段

prepare阶段、optimize阶段、execute阶段

prepare阶段

- 检查表和字段是否存在
- 将*符号换成表中所有字段

optimize阶段

优化器为sql查询语句制定一个执行计划

例如执行时选择哪个索引

execute阶段

根据执行计划执行SQL查询语句，从存储引擎读取记录，返回给客户端

事务有哪些特性?

- 原子性
  - 一个事务中的所有操作，要么全部完成，要么全部不完成
  - 通过undo log（回滚日志）来保证的
- 一致性
  - 事务操作前和操作后，数据满足完整性约束，数据库保持一致性状态
  - 通过 原子性 、 隔离性 和 持久性 来保证的
- 隔离性
  - 数据库允许多个并发事务同时对数据进行读写和修改
  - 通过MVCC（多版本并发控制）或锁机制来保证的
- 持久性
  - 事务处理结束后，对数据的修改是永久的，即使系统故障也不会丢失
  - 通过redo log（重做日志）来保证的

并行事务会引发什么问题?

- 脏读
  - 一个事务读到了另一个未提交事务修改过的数据
- 不可重复读
  - 一个事务多次读取同一个数据，出现了前后数据不一致的情况
- 幻读
  - 一个事务内多次查询符合同一查询条件的记录数量，出现前后记录数量不一致的情况

脏读最严重，往下依次递减

事务的隔离级别有哪些?

隔离级别越高，性能效率越低

- 读未提交
  - 一个事务未提交时，它所做的变更就可以被其他事务看见
  - 脏读、不可重复读、幻读
- 读提交
  - 一个事务提交了，它所做的变更才能被其他事务看见
  - 不可重复读、幻读

- 可重复读
  - 一个事务执行过程中所看到的数据，与它事务开始时看见的数据是一致的
  - 是`InnoDB`引擎默认的隔离级别
  - 幻读
- 串行化
  - 会对记录加上读写锁，当多个事务对同一个记录进行读写操作时，如果发生了读写冲突，后一个事务需要等前一个事务执行完成

InnoDB的默认隔离级别虽然是可重复读，但它可以很大程度上避免幻读现象，解决方案有两种：

- 针对快照读，是通过MVCC来解决幻读问题的
- 针对当前读（select .... for update），是通过next-key lock（记录锁+间隙锁）来解决的，当执行 select ... for update 时，会加上netx-key lock锁，如果其他事务在next-key lock锁范围内插入了一条数据，那么这个插入语句会被阻塞

四种隔离级别具体是如何实现的?

- 读未提交，就是直接读取最新数据
- 串行化，是通过加读写锁来避免并行访问
- 读提交和可重复读，都是通过Read View来实现的，只是创建Read View的时机不同
  - 读提交的是在每个语句执行前创建
  - 可重复读是在启动事务时创建

Mysql有两种开始事务的命令：

- begin/start transaction
  - 执行该命令后，执行了第一条select语句，才算事务真正启动
- start transaction with consistent snapshot
  - 执行该命令后，立刻启动

Read View是如何在MVCC工作的?

Read View中有四个字段

- creator_trx_id：创建该Read View的事务的事务id
- m_ids：创建Read View时，数据库处于活跃且暂未提交状态的事务id列表

- min_trx_id：创建Read View时，处于活跃且暂未提交状态的事务中的最小事务id
- max_trx_id：创建Read View时，数据库分配给下一个事务的事务id

聚簇索引记录中的两个隐藏列

- trx_id：当事务对聚簇索引记录进行修改时，就会把该事务id记录在trx_id隐藏列中
- roll_pointer：每次对聚簇索引记录进行改动时，都会把旧版本的记录写入undo日志中，roll_pointer就是一个指向该旧版本记录的指针

当一个事务访问记录时，会有以下三种情况：

- 访问的记录的trx_id小于min_trx_id，表示这个版本的记录是在创建Read View之前已提交的事务生成的，该版本记录对当前事务可见
- 访问的记录的trx_id大于等于max_trx_id，表示这个版本的记录是创建Read View之后才启动的事务生成的，该版本记录对当前事务不可见
- 访问的记录的trx_id在min_trx_id和max_trx_id之间，需要查看trx_id是否在m_ids列表中
    - 在列表中，表示生成该版本记录的事务仍然活跃着，该版本记录对该事务不可见
    - 否则表示生成该版本记录的事务已提交，可见

MVCC多版本并发控制就是通过版本链来控制并发事务对同一个记录的访问

Mysql有哪些锁?

- 全局锁
- 表级锁
- 行级锁

全局锁

全局锁怎么用?

使用全局锁

```
flush tables with read lock
```

数据库将处于只读状态

释放全局锁

```
unlock tables
```

会话断开也会释放

应用场景

全库逻辑备份

缺点

业务停滞

既然备份数据库时使用全局锁会影响业务，有什么方法可以避免吗?

如果数据库的存储引擎支持可重复读的隔离级别，可以在备份前开启事务，其他事务可以正常进行，备份事务期间都使用Read View，备份数据不会改变

备份数据库的工具是mysqldump，在使用mysqldump时加上-single-transaction，就可以实现备份数据前开启事务

这种方法只适用于支持可重复读隔离级别的存储引擎，InnoDB可以，MyISAM不行

表级锁

有哪些?

- 表锁
- 元数据锁（MDL）
- 意向锁
- AUTO-INC锁

表锁

当前会话和其他会话可以对该表读，不能写
当前会话不能访问其他表

```
lock tables student read // 表级共享锁
```

当前会话可以对该表进行读写，其他会话读写都不可

当前会话不能访问其他表

```
lock tables student write // 表级排他锁
```

解锁

```
unlock tables
```

元数据锁MDL

当我们对数据库表操作时，会自动加元数据锁

增删查改->元数据读锁

对表结构进行修改->元数据写锁

MDL不需要显示调用，那它什么时候被释放?

事务提交后会释放

1. 如果有一个select事务，长时间不提交，MDL读锁一直没被释放
2. 其他select事务不影响，读读不冲突
3. 此时修改表结构，MDL写锁阻塞
4. 其他select事务会阻塞，读写冲突

所以，在修改表结构前，需要查看是否有事务加上了MDL读锁，如果有，最好kill掉再修改

意向锁

在使用 InnoDB 引擎的表里对某些记录加上「共享锁」之前，需要先在表级别加上一个「意向共享锁」；

在使用 InnoDB 引擎的表里对某些纪录加上「独占锁」之前，需要先在表级别加上一个「意向独占锁」；

意向共享锁和意向独占锁是表级锁，不会和行级的共享锁和独占锁发生冲突，而且意向锁之间也不会发生冲突，只会和共享表锁（lock tables ... read）和独占表锁（lock tables ... write）发生冲突。

表锁和行锁是满足读读共享、读写互斥、写写互斥的。

如果没有「意向锁」，那么加「独占表锁」时，就需要遍历表里所有记录，查看是否有记录存在独占锁，这样效率会很慢。 那么有了「意向锁」，由于在对记录加独占锁前，先会加上表级别的意向独占锁，那么在加「独占表锁」时，直接查该表是否有意向独占锁，如果有就意味着表里已经有记录被加了独占锁，这样就不用去遍历表里的记录。

所以，意向锁的目的是为了快速判断表里是否有记录被加锁

AUTO-INC锁

数据库会自动给主键赋值递增的值，这主要是通过 AUTO-INC 锁实现的。

AUTO-INC 锁是特殊的表锁机制，锁不是再一个事务提交后才释放，而是再执行完插入语句后就会立即释放。

在插入数据时，会加一个表级别的 AUTO-INC 锁，然后为被 AUTO_INCREMENT 修饰的字段赋值递增的值，等插入语句执行完成后，才会把 AUTO-INC 锁释放掉。

那么，一个事务在持有 AUTO-INC 锁的过程中，其他事务的如果要向该表插入语句都会被阻塞，从而保证插入数据时，被 AUTO_INCREMENT 修饰的字段的值是连续递增的。 但是， AUTO-INC 锁再对大量数据进行插入的时候，会影响插入性能，因为另一个事务中的插入会被阻塞。

因此， 在 MySQL 5.1.22 版本开始，InnoDB 存储引擎提供了一种轻量级的锁来实现自增。 一样也是在插入数据的时候，会为被 AUTO_INCREMENT 修饰的字段加上轻量级锁，然后给该字段赋值一个自增的值，就把这个轻量级锁释放了，而不需要等待整个插入语句执行完后才释放锁。

InnoDB 存储引擎提供了个 innodb_autoinc_lock_mode 的系统变量，是用来控制选择用 AUTO-INC 锁，还是轻量级的锁。

- 当 innodb_autoinc_lock_mode = 0，就采用 AUTO-INC 锁，语句执行结束后才释放锁；
- 当 innodb_autoinc_lock_mode = 2，就采用轻量级锁，申请自增主键后就释放锁，并不需要等语句执行后才释放。
- 当 innodb_autoinc_lock_mode = 1： 普通 insert 语句，自增锁在申请之后就马上释放； 类似 insert … select 这样的批量插入数据的语句，自增锁还是要等语句结束后才被释放；

当 innodb_autoinc_lock_mode = 2 是性能最高的方式，但是当搭配 binlog 的日志格式是 statement 一起使用的时候，在「主从复制的场景」中会发生数据不一致的问题

行级锁

InnoDB支持行级锁，MyISAM不支持

快照读，也就是普通的select语句是不会对记录加锁的
锁定读，需要用下面这两个方式，且用的时候需要在同一个事务中

```
select ... lock in share mode
select ... for update
```

行级锁的类型

- Record Lock： 记录锁，只锁定一条记录
- Gap Lock： 间隙锁，锁定一个范围，不含记录本身
- Next-key Lock ： 记录锁和间隙锁的组合，锁定一个范围，并锁定记录本身

Record Lock

分为S型记录锁和X型记录锁

一个事务对一条记录加S锁后，其他事务可以对他加S锁，不可以加X锁

一个事务对一条记录加X锁后，其他事务不可以加S、X锁

Gap Lock

间隙锁，存在于可重复读的隔离级别和串行化的隔离级别，目的是解决可重复读出现的幻读现象

例如，范围为（3,5）的间隙锁，不能插入id为4的记录

间隙锁也有S和X锁，但没啥区别，间隙锁之间是兼容的，也就是说不同事务可以同时持有相同范围的间隙锁

Next-Key Lock

临键锁

例如，范围为（3，5] 的临键锁，不能插入4和5

插入意向锁

一个事务在执行插入事务前，会先查看插入的位置是否被加了间隙锁，如果存在间隙锁，当前事务阻塞并生成一个插入意向锁，表示有事务想在区间内插入数据，但目前处于等待状态

什么SQL语句会加行级锁?

```
select ... lock in share mode // 共享锁
select ... for update // 独占锁
update 和 delete // 独占锁
```

Mysql是怎么加行级锁的?

加锁的对象是索引，加锁的基本单位是next-key lock

临键锁是左开右闭区间

间隙锁是左开右开区间

什么情况下next-key lock 会退化为记录锁或间隙锁?

在使用记录锁或者间隙锁就能避免幻读现象的情况下，就会退化

唯一索引（主键索引）等值查询

当我们用唯一索引进行等值查询的时候，查询的记录存不存在，加锁的规则也会不同：

- 当查询的记录是「存在」的，在索引树上定位到这一条记录后，将该记录的索引中的 next-key lock 会退化成「记录锁」。
- 当查询的记录是「不存在」的，在索引树找到第一条大于该查询记录的记录后，将该记录的索引中的 next-key lock 会退化成「间隙锁
    - 假如查id=2，不存在，此时表中有id=1、id=5....，next-key lock 会退化为（1,5）的间隙锁，防止其他事务插入2、3、4的记录产生幻读

字节面试：加了什么锁，导致死锁的?

字节面试：加了什么锁，导致死锁的？ | 小林coding | Java面试学习

Mysql有哪几种日志?

- undo log （回滚日志）：是InnoDB存储引擎层生成的日志，实现了事务的原子性，主要用于事务回滚和MVCC
- redo log （重做日志）：是InnoDB存储引擎层生成的日志，实现了事务的持久性，主要用于掉电等故障恢复
- binlog （归档日志）：是Server层生成的日志，主要用于数据备份和主从复制

为什么需要undo log?

当我们增删改时，Mysql会隐性的开启事务，执行完一条语句是否提交事务，是由参数autocommit决定的，默认是开启的

如果一个事务运行过程中，Mysql发生了崩溃，事务还未提交，怎么回滚到事务开启前的数据呢?

如果我们在事务执行过程中，记录回滚时需要用到的信息到日志中，实现这一机制的就是undo log

- 当插入记录时，将记录的主键值记录下来，回滚时删除对应主键值的记录即可
- 当删除记录时，将记录的完整信息记录下来，回滚时新增记录即可
- 当修改记录时，将被更新列的旧值记录下来，回滚时更新回去即可

undo log 两大作用:

- 实现事务回滚，保障事务的原子性。事务处理过程中，如果出现了错误或者用户执 行了 ROLLBACK 语句，MySQL 可以利用 undo log 中的历史数据将数据恢复到事务开始之前的状态。
- 实现 MVCC（多版本并发控制）关键因素之一。MVCC 是通过 ReadView + undo log 实现的。undo log 为每条记录保存多份历史数据，MySQL 在执行快照读（普通 select 语句）的时候，会根据事务的 Read View 里的信息，顺着 undo log 的版本链找到满足其可见性的记录

为什么需要Buffer Pool?

Mysql的数据都是存在磁盘中的，当我们需要更新数据时，先要把磁盘中的数据拿出来，放到内存中，再将内存中的数据修改，修改后的数据是直接写回磁盘好呢？还是放在内存好呢?

放在内存好，下次要查询此数据直接从内存拿即可

为此，InnoDB专门设计了一个缓冲池（Buffer Pool），来提高数据库的性能

有了Buffer Pool之后:

- 当需要查数据时，数据如果存在缓冲池，直接读缓冲池就可
- 当需要改数据时，如果数据在缓冲池，将缓冲池中的**数据所对应的数据页**修改，并标记为**脏页**即可，以后再挑选一个合适的时机由**后台线程**写回

为什么需要redo log?

因为Buffer Pool是基于内存的，一旦断电，数据就会丢失

为了防止这种情况，当一条记录需要更新时，InnoDB会先更新内存，再将此数据页的修改信息写入redo log中

什么是redo log?

redo log是物理日志，记录了某个数据页做了什么修改

事务提交时，只需要将redo log持久化到磁盘即可

被修改 Undo 页面，需要记录对应 redo log 吗?

需要的。 开启事务后，InnoDB 层更新记录前，首先要记录相应的 undo log，如果是更新操作，需要把被更新的列的旧值记下来，也就是要生成一条 undo log，undo log 会写入 Buffer Pool 中的 Undo 页面。 不过，在内存修改该 Undo 页面后，也是需要记录对应的 redo log，因为undo log也要实现持久性的保护

redo log 和 undo log 区别在哪?

这两种日志是属于 InnoDB 存储引擎的日志，它们的区别在于：

redo log 记录了此次事务「修改后」的数据状态，记录的是更新之后的值，主要用于事务崩溃恢复，保证事务的持久性。

undo log 记录了此次事务「修改前」的数据状态，记录的是更新之前的值，主要用于事务回滚，保证事务的原子性。

所以有了 redo log，再通过 WAL 技术，InnoDB 就可以保证即使数据库发生异常重启，之前已提交的记录都不会丢失，这个能力称为 crash-safe（崩溃恢复）。可以看出来， redo log 保证了事务四大特性中的持久性

redo log 要写到磁盘，数据也要写磁盘，为什么要多此一举?

写入 redo log 的方式使用了追加操作， 所以磁盘操作是顺序写，而写入数据需要先找到写入位置，然后才写到磁盘，所以磁盘操作是随机写。

磁盘的「顺序写 」比「随机写」 高效的多，因此 redo log 写入磁盘的开销更小

产生的 redo log 是直接写入磁盘的吗?

不是的。 实际上， 执行一个事务的过程中，产生的 redo log 也不是直接写入磁盘的，因为这样会产生大量的 I/O 操作，而且磁盘的运行速度远慢于内存。

所以，redo log 也有自己的缓存—— redo log buffer，每当产生一条 redo log 时，会先写入到 redo log buffer

redo log什么时候刷盘?

缓存在 redo log buffer 里的 redo log 还是在内存中，它什么时候刷新到磁盘?

主要有下面几个时机:

- MySQL 正常关闭时;
- 当 redo log buffer 中记录的写入量大于 redo log buffer 内存空间的一半时，会触发落盘;
- InnoDB 的后台线程每隔 1 秒，将 redo log buffer 持久化到磁盘。
- 每次事务提交时都将缓存在 redo log buffer 里的 redo log 直接持久化到磁盘（这个策略可由 innodb_flush_log_at_trx_commit 参数控制，下面会说）。

> innodb_flush_log_at_trx_commit 参数控制的是什么?

单独执行一个更新语句的时候，InnoDB 引擎会自己启动一个事务，在执行更新语句的过程中，生成的 redo log 先写入到 redo log buffer 中，然后等事务提交的时候，再将缓存在 redo log buffer 中的 redo log 按组的方式「顺序写」到磁盘。

上面这种 redo log 刷盘时机是在事务提交的时候，这个默认的行为。

除此之外，InnoDB 还提供了另外两种策略，由参数 `innodb_flush_log_at_trx_commit` 参数控制，可取的值有：0、1、2，默认值为 1，这三个值分别代表的策略如下:

- 当设置该**参数为 0 时**，表示每次事务提交时 ，还是**将 redo log 留在 redo log buffer 中** ，该模式下在事务提交时不会主动触发写入磁盘的操作。
- 当设置该**参数为 1 时**，表示每次事务提交时，都**将缓存在 redo log buffer 里的 redo log 直接持久化到磁盘**，这样可以保证 MySQL 异常重启之后数据不会丢失。
- 当设置该**参数为 2 时**，表示每次事务提交时，都只是缓存在 redo log buffer 里的 redo log **写到 redo log 文件，注意写入到「 redo log 文件」并不意味着写入到了磁盘**，因为操作系统的文件系统中有个 Page Cache（如果你想了解 Page Cache，可以看这篇 ），Page Cache 是专门用来缓存文件数据的，所以写入「 redo log文件」意味着写入到了操作系统的文件缓存。

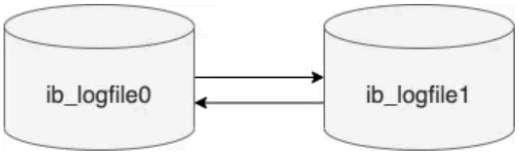
innodb_flush_log_at_trx_commit 为 0 和 2 的时候，什么时候才将 redo log 写入磁盘?

InnoDB 的后台线程每隔 1 秒：

- 针对参数 0 ： 会把缓存在 redo log buffer 中的 redo log ，通过调用 `write()` 写到操作系统的 Page Cache，然后调用 `fsync()` 持久化到磁盘。**所以参数为 0 的策略，MySQL 进程的崩溃会导致上一秒钟所有事务数据的丢失**;
- 针对参数 2 ： 调用 fsync，将缓存在操作系统中 Page Cache 里的 redo log 持久化到磁盘。**所以参数为 2 的策略，较取值为 0 情况下更安全，因为 MySQL 进程的崩溃并不会丢失数据，只有在操作系统崩溃或者系统断电的情况下，上一秒钟所有事务数据才可能丢失**。

redo log 文件写满了怎么办?

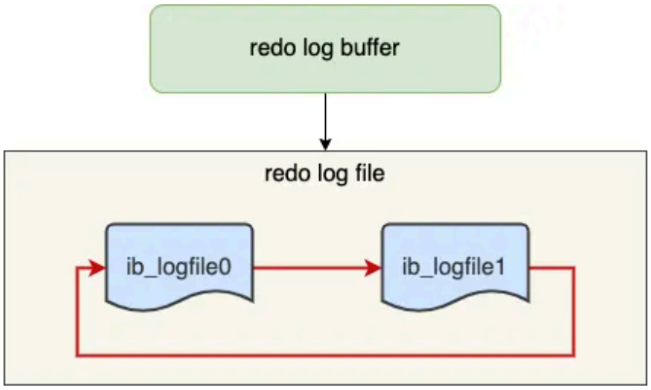
默认情况下， InnoDB 存储引擎有 1 个重做日志文件组(redo log Group)，「重做日志文件组」由有 2 个 redo log 文件组成，这两个 redo 日志的文件名叫： `ib_logfile0` 和 `ib_logfile1` 。



在重做日志组中，每个 redo log File 的大小是固定且一致的，假设每个 redo log File 设置的上限是 1 GB，那么总共就可以记录 2GB 的操作。

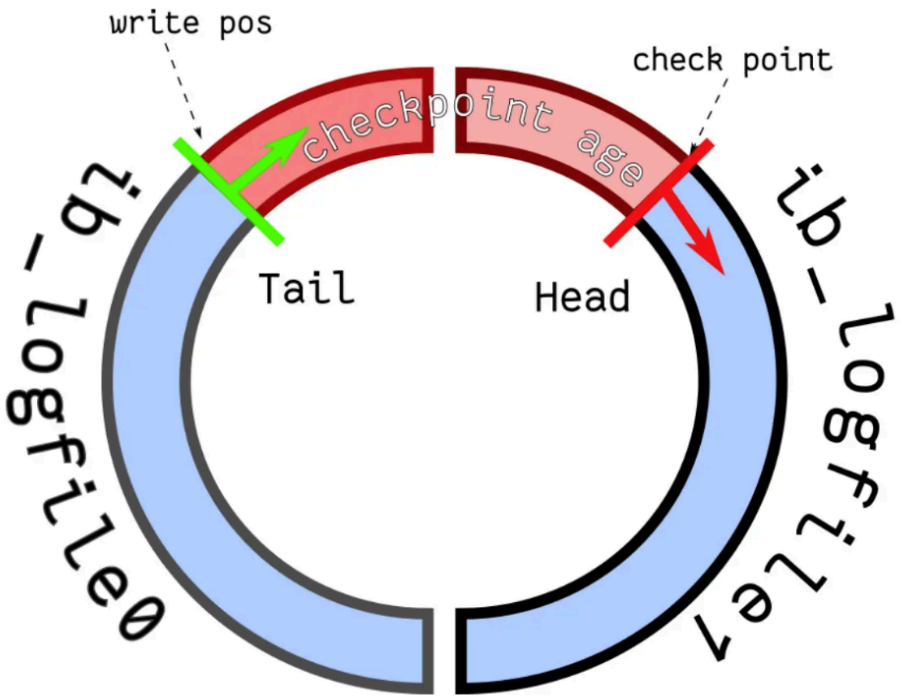
重做日志文件组是以**循环写**的方式工作的，从头开始写，写到末尾就又回到开头，相当于一个环形。

所以 InnoDB 存储引擎会先写 ib_logfile0 文件，当 ib_logfile0 文件被写满的时候，会切换至 ib_logfile1 文件，当 ib_logfile1 文件也被写满时，会切换回 ib_logfile0 文件。



我们知道 redo log 是为了防止 Buffer Pool 中的脏页丢失而设计的，那么如果随着系统运行，Buffer Pool 的脏页刷新到了磁盘中，那么 redo log 对应的记录也就没用了，这时候我们擦除这些旧记录，以腾出空间记录新的更新操作。

redo log 是循环写的方式，相当于一个环形，InnoDB 用 write pos 表示 redo log 当前记录写到的位置，用 checkpoint 表示当前要擦除的位置，如下图：



图中的：

- write pos 和 checkpoint 的移动都是顺时针方向；
- write pos ~ checkpoint 之间的部分（图中的红色部分），用来记录新的更新操作；
- check point ~ write pos 之间的部分（图中蓝色部分）：待落盘的脏数据页记录；

如果 write pos 追上了 checkpoint，就意味着 **redo log 文件满了，这时 MySQL 不能再执行新的更新操作，也就是说 MySQL 会被阻塞**（*因此所以针对并发量大的系统，适当设置 redo log 的文件大小非常重要*），此时**会停下来将 Buffer Pool 中的脏页刷新到磁盘中，然后标记 redo log 哪些记录可以被擦除，接着对旧的 redo log 记录进行擦除，等擦除完旧记录腾出了空间，checkpoint 就会往后移动（图中顺时针）**，然后 MySQL 恢复正常运行，继续执行新的更新操作。

所以，一次 checkpoint 的过程就是脏页刷新到磁盘中变成干净页，然后标记 redo log 哪些记录可以被覆盖的过程。

binlog是什么?

binlog是server层生成的

它记录了所有数据库表结构变更和表数据修改的日志

redo log和binlog的区别

1. 适用对象不同

redo log只适用于InnoDB

binlog适用于所有存储引擎，因为它是在server层

2. 文件格式不同

3. 写入方式不同

redo log是循环写，写满就重头写

binlog是追加写，写满就创建新的文件

4. 用途不同

redo log 用于断电故障恢复

binlog用于备份恢复和主从复制

> 如果不小心整个数据库的数据被删除了，能使用 redo log 文件恢复数据吗?
>
> 不可以使用 redo log 文件恢复，只能使用 binlog 文件恢复。
>
> 因为 redo log 文件是循环写，是会边写边擦除日志的，只记录未被刷入磁盘的数据的物理日志，已经刷入磁盘的数据都会从 redo log 文件里擦除。
>
> binlog 文件保存的是全量的日志，也就是保存了所有数据变更的情况，理论上只要记录在 binlog 上的数据，都可以恢复，所以如果不小心整个数据库的数据被删除了，得用 binlog 文件恢复数据。

主从复制是如何实现的?

主从复制是依赖binlog实现的，也就是将数据库上发生的所有变化以二进制的形式保存到磁盘，复制也就是将binlog的数据从主库传输到从库上

这个复制过程是异步的，也就是说主库上的事务不会阻塞等待复制binlog的线程同步完成

主从复制的流程如下：

主库写入binlog，从库创建一个专门的IO线程连接主库的log dump线程，接收binlog并写入relay log的中继日志中，从库再创建一个线程从relay log的中继日志中读取binlog来回放

从库是不是越多越好?

不是，从库越多，从库连接上来的IO线程越多，主库也需要提供相同数量的log dump线程，对主库资源消耗较高

Mysql主从复制还有哪些模型?

- 同步复制：提交事务的线程需要等待所有从库的复制成功响应，才能返回客户端结果
- 异步复制：不需要等待从库复制成功，直接返回客户端结果
- 半同步复制：事务线程不用等待所有从库复制成功，只需一半从库复制成功，就可以返回客户端结果

binlog什么时候刷盘?

事务执行时，先把日志写到binlog cache中，事务提交时，再把binlog cache写到binlog文件中

binlog cache每个线程都有一份

binlog文件是操作系统文件缓存，还需要持久化到磁盘

Mysql提供sync_binlog来控制binlog持久化到磁盘上的频率

- sync_binlog=0：每次事务提交都只write（binlog cache写到binlog文件），什么时候fsync（binlog文件持久化到磁盘）由操作系统决定
- sync_binlog=1：每次事务提交先write，然后马上fsync
- sync_binlog=N（N>1）：每次事务提交都write，累计N个事务后fsync

一般设置为100~1000中的某个数值

update语句执行流程

当优化器分析出成本最小的执行计划后，执行器就按照执行计划开始进行更新操作。

具体更新一条记录 `UPDATE t_user SET name = 'xiaolin' WHERE id = 1;` 的流程如下:

1. 执行器负责具体执行，会调用存储引擎的接口，通过主键索引树搜索获取 id = 1 这一行记录：
   - 如果 id=1 这一行所在的数据页本来就在 buffer pool 中，就直接返回给执行器更新；
   - 如果记录不在 buffer pool，将数据页从磁盘读入到 buffer pool，返回记录给执行器。
2. 执行器得到聚簇索引记录后，会看一下更新前的记录和更新后的记录是否一样：
   - 如果一样的话就不进行后续更新流程；
   - 如果不一样的话就把更新前的记录和更新后的记录都当作参数传给 InnoDB 层，让 InnoDB 真正的执行更新记录的操作；
3. 开启事务， InnoDB 层更新记录前，首先要记录相应的 undo log，因为这是更新操作，需要把被更新的列的旧值记下来，也就是要生成一条 undo log，undo log 会写入 Buffer Pool 中的 Undo 页面，不过在内存修改该 Undo 页面后，需要记录对应的 redo log。
4. InnoDB 层开始更新记录，会先更新内存（同时标记为脏页），然后将记录写到 redo log 里面，这个时候更新就算完成了。为了减少磁盘I/O，不会立即将脏页写入磁盘，后续由后台线程选择一个合适的时机将脏页写入到磁盘。这就是 **WAL 技术**，MySQL 的写操作并不是立刻写到磁盘上，而是先写 redo 日志，然后在合适的时间再将修改的行数据写到磁盘上。
5. 至此，一条记录更新完了。
6. 在一条更新语句执行完成后，然后开始记录该语句对应的 binlog，此时记录的 binlog 会被保存到 binlog cache，并没有刷新到硬盘上的 binlog 文件，在事务提交时才会统一将该事务运行过程中的所有 binlog 刷新到硬盘。
7. 事务提交，剩下的就是「两阶段提交」的事情了，接下来就讲这个。

为什么需要两阶段提交?

事务提交后，redo log 和 binlog 都要持久化到磁盘，但是这两个是独立的逻辑，可能出现半成功的状态，这样就造成两份日志之间的逻辑不一致。

举个例子，假设 id = 1 这行数据的字段 name 的值原本是 'jay'，然后执行 `UPDATE t_user SET name = 'xiaolin' WHERE id = 1;` 如果在持久化 redo log 和 binlog 两个日志的过程中，出现了半成功状态，那么就有两种情况：

- **如果在将 redo log 刷入到磁盘之后， MySQL 突然宕机了，而 binlog 还没有来得及写入**。MySQL 重启后，通过 redo log 能将 Buffer Pool 中 id = 1 这行数据的 name 字段恢复到新值 xiaolin，但是 binlog 里面没有记录这条更新语句，在主从架构中，binlog 会被复制到从库，由于 binlog 丢失了这条更新语句，从库的这一行 name 字段是旧值 jay，与主库的值不一致性；
- **如果在将 binlog 刷入到磁盘之后， MySQL 突然宕机了，而 redo log 还没有来得及写入**。由于 redo log 还没写，崩溃恢复以后这个事务无效，所以 id = 1 这行数据的 name 字段还是旧值 jay，而 binlog 里面记录了这条更新语句，在主从架构中，binlog 会被复制到从库，从库执行了这条更新语句，那么这一行 name 字段是新值 xiaolin，与主库的值不一致性；

可以看到，在持久化 redo log 和 binlog 这两份日志的时候，如果出现半成功的状态，就会造成主从环境的数据不一致性。这是因为 redo log 影响主库的数据，binlog 影响从库的数据，所以 redo log 和 binlog 必须保持一致才能保证主从数据一致。

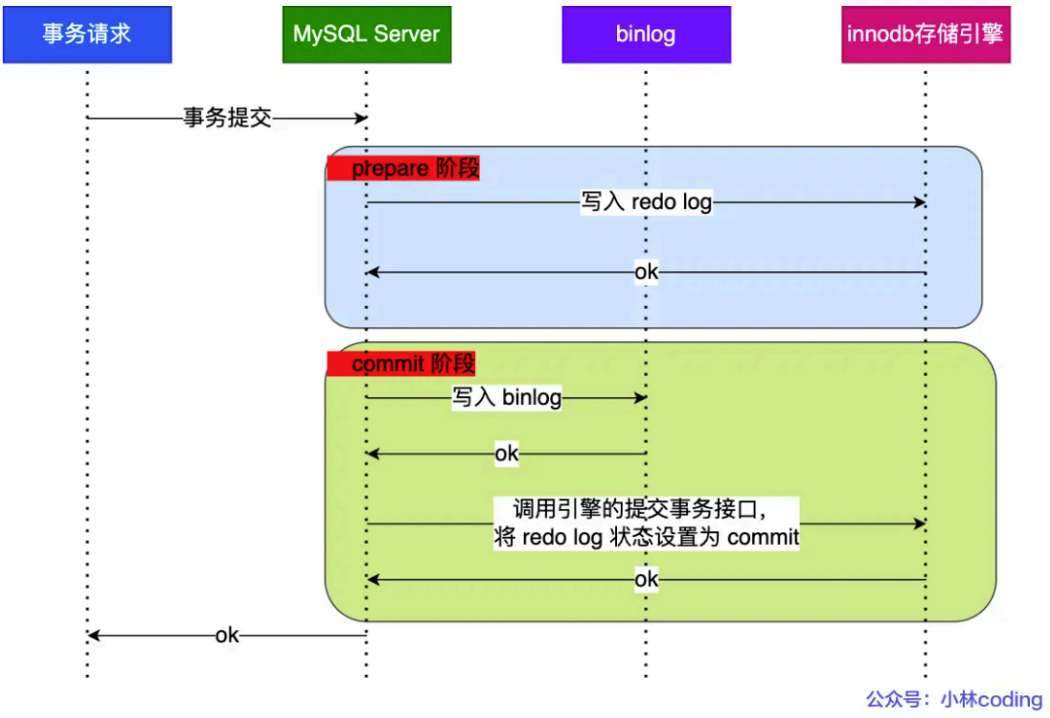
MySQL 为了避免出现两份日志之间的逻辑不一致的问题，使用了「两阶段提交」来解决，两阶段提交其实是分布式事务一致性协议，它可以保证多个逻辑操作要不全部成功，要不全部失败，不会出现半成功的状态。

两阶段提交把单个事务的提交拆分成了 2 个阶段，分别是「准备（Prepare）阶段」和「提交（Commit）阶段」，每个阶段都由协调者（Coordinator）和参与者（Participant）共同完成。注意，不要把提交（Commit）阶段和 commit 语句混淆了，commit 语句执行的时候，会包含提交（Commit）阶段。

两阶段提交的过程

在 MySQL 的 InnoDB 存储引擎中，开启 binlog 的情况下，MySQL 会同时维护 binlog 日志与 InnoDB 的 redo log，为了保证这两个日志的一致性，MySQL 使用了**内部 XA 事务**（是的，也有外部 XA 事务，跟本文不太相关，我就不介绍了），内部 XA 事务由 binlog 作为协调者，存储引擎是参与者。

当客户端执行 commit 语句或者在自动提交的情况下，MySQL 内部开启一个 XA 事务，**分两阶段来完成 XA 事务的提交**，如下图：

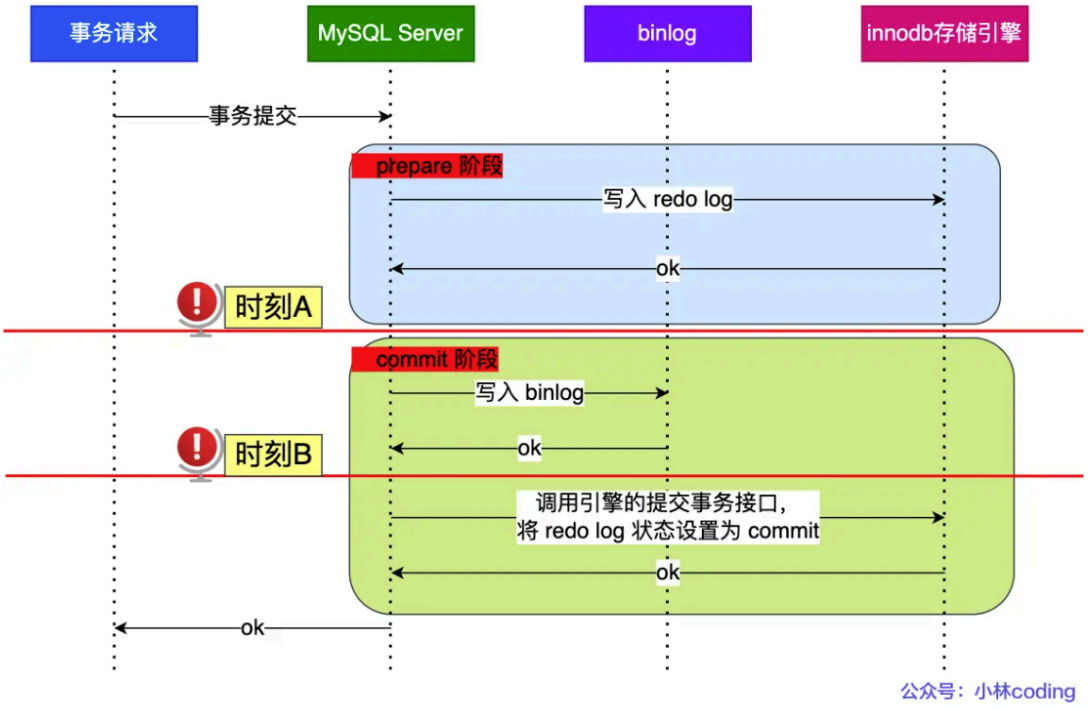


从图中可看出，事务的提交过程有两个阶段，就是**将 redo log 的写入拆成了两个步骤：prepare 和 commit，中间再穿插写入binlog**，具体如下：

- **prepare 阶段**：将 XID（内部 XA 事务的 ID） 写入到 redo log，同时将 redo log 对应的事务状态设置为 prepare，然后将 redo log 持久化到磁盘（innodb_flush_log_at_trx_commit = 1 的作用）；
- **commit 阶段**：把 XID 写入到 binlog，然后将 binlog 持久化到磁盘（sync_binlog = 1 的作用），接着调用引擎的提交事务接口，将 redo log 状态设置为 commit，此时该状态并不需要持久化到磁盘，只需要 write 到文件系统的 page cache 中就够了，因为只要 binlog 写磁盘成功，就算 redo log 的状态还是 prepare 也没有关系，一样会被认为事务已经执行成功；

异常重启会出现什么情况?

我们来看看在两阶段提交的不同时刻，MySQL 异常重启会出现什么现象？下图中有时刻 A 和时刻 B 都有可能发生崩溃：



不管是时刻 A（redo log 已经写入磁盘， binlog 还没写入磁盘），还是时刻 B （redo log 和 binlog 都已经写入磁盘，还没写入 commit 标识）崩溃，**此时的 redo log 都处于 prepare 状态**。

在 MySQL 重启后会按顺序扫描 redo log 文件，碰到处于 prepare 状态的 redo log，就拿着 redo log 中的 XID 去 binlog 查看是否存在此 XID：

- **如果 binlog 中没有当前内部 XA 事务的 XID，说明 redolog 完成刷盘，但是 binlog 还没有刷盘，则回滚事务**。对应时刻 A 崩溃恢复的情况。
- **如果 binlog 中有当前内部 XA 事务的 XID，说明 redolog 和 binlog 都已经完成了刷盘，则提交事务**。对应时刻 B 崩溃恢复的情况。

可以看到，**对于处于 prepare 阶段的 redo log，即可以提交事务，也可以回滚事务，这取决于是否能在 binlog 中查找到与 redo log 相同的 XID**，如果有就提交事务，如果没有就回滚事务。这样就可以保证 redo log 和 binlog 这两份日志的一致性了。

所以说，**两阶段提交是以 binlog 写成功为事务提交成功的标识**，因为 binlog 写成功了，就意味着能在 binlog 中查找到与 redo log 相同的 XID。

> 处于 prepare 阶段的 redo log 加上完整 binlog，重启就提交事务，MySQL 为什么要这么设计?

binlog 已经写入了，之后就会被从库（或者用这个 binlog 恢复出来的库）使用。

所以，在主库上也要提交这个事务。采用这个策略，主库和备库的数据就保证了一致性。

为什么 `prepare` 状态的 redo log 加上完整的 binlog 也要提交?

- **逻辑一致性**：因为 binlog 一旦写完并落盘，它就会被从库（Slave）读取并执行。
- **必须提交**：如果此时主库重启后因为 redo log 是 `prepare` 就选择回滚，那么主库就没有这笔数据，而从库却有了。

- **结论**：为了保证主从数据绝对一致，只要 binlog 写入成功了，主库重启后就必须把对应的 `prepare` 事务提交掉。